AI Response

Provider: openai
Model: gpt-5
Original: START_testnewmodeloutputrendering_openai.docx
Generated: 2026-09-03T03:45:10.931Z

---

a) Intended function and scheduling

- Function: A background sensor thread reads keyboard input ('e' or 'n') and writes the latest event into a shared structure (key, count). The main thread runs one step of a traffic-light state machine per loop and reacts to the sensor by advancing from East–West green to North–South phases when 'n' is detected.
- Intended scheduling: The program comment and attribute setup indicate the sensor thread should run round-robin at priority 20 so it can preempt the main thread and minimize sensing latency. In QNX, scheduling decisions preempt a running lower-priority thread when a higher-priority thread becomes READY; round-robin time-slices only apply among threads of equal priority.

b) Three defects and their runtime consequences (single-CPU)

1. Scheduling attributes don't take effect (missing PTHREAD_EXPLICIT_SCHED)

- Defect: The code sets SCHED_RR and priority 20 in the attributes but never calls pthread_attr_setinheritsched(..., PTHREAD_EXPLICIT_SCHED). With the default inherit setting, the created thread inherits the parent's policy/priority, and the requested settings are ignored.
- Consequence: The sensor thread doesn't run at the intended higher priority. On a single CPU, if it shares the main thread's priority, it can't reliably preempt the main thread. Depending on the policy, it runs only on time slices or when the main thread blocks/yields, increasing sensor latency and risking starvation if FIFO applies.

2. Data race on the shared structure

- Defect: Both threads read/write shared.key and shared.count without synchronization.
- Consequence: Undefined behavior. The main thread may never observe the sensor's updates (e.g., the compiler can hoist shared.key into a register inside the empty while loop), leading to an apparent "stuck" green. Updates to count can be lost, and a concurrent write by the sensor can be lost when the main thread clears key to 0. QNX documentation requires synchronization (e.g., mutexes/condvars) for safe shared-memory access between threads.

3. Busy-waiting on the sensor flag

- Defect: The state EWG_NSR spins in while (shared.key != 'n') with an empty body.
- Consequence: Wastes CPU and, combined with defect (1), can prevent the sensor thread from running promptly on a single CPU because the running thread continues unless it blocks, is preempted by a higher-priority thread, or yields. This degrades responsiveness and determinism; the correct approach is to block on a synchronization primitive signaled by the sensor.

c) Corrections (clearly labelled snippets)

1. Make the sensor thread actually run at the intended policy/priority

- Add explicit scheduling inheritance control before pthread_create:

// ensure attributes are applied rather than inherited pthread_attr_setinheritsched(&sensor_attr, PTHREAD_EXPLICIT_SCHED);

2. Protect the shared structure and avoid busy-wait with a mutex + condition variable

- Declarations (global scope, alongside shared):

pthread_mutex_t shared_mtx = PTHREAD_MUTEX_INITIALIZER; pthread_cond_t sensor_cv = PTHREAD_COND_INITIALIZER;

- Sensor thread: update under the mutex and signal waiters

void *sensor_thread(void *arg) { int c; while (1) { c = getchar(); if (c == 'e' || c == 'n') { pthread_mutex_lock(&shared_mtx); shared.key = (char)c; shared.count = shared.count + 1; pthread_cond_signal(&sensor_cv); // notify main pthread_mutex_unlock(&shared_mtx); } } return NULL; }

- State machine (EWG_NSR case): replace spin with blocking wait; clear the flag while holding the lock

case EWG_NSR: printf("East-West GREEN , North-South RED\n"); pthread_mutex_lock(&shared_mtx); while (shared.key != 'n') { pthread_cond_wait(&sensor_cv, &shared_mtx); } shared.key = 0; // consume the event pthread_mutex_unlock(&shared_mtx); *CurState = EWY_NSR; break;

These changes:

- Apply the intended SCHED_RR/priority 20 to the sensor thread.
- Remove the data race and prevent missed/phantom reads of shared.key.
- Eliminate CPU-burning polling and ensure prompt wake-up when a sensor event arrives.

d) Priority inversion and the better primitive

- Priority inversion: A higher-priority thread blocks on a resource held by a lower-priority thread, while medium-priority threads preempt the lower-priority one, delaying the release and indirectly blocking the higher-priority thread longer than necessary.
- Choice: A QNX mutex is preferable to an unnamed semaphore for protecting the shared structure in a priority-driven real-time system. Semaphores don’t adjust effective priority; if you need priority inheritance, use a mutex. Mutexes can raise the holder’s effective priority to that of the waiting higher-priority thread, mitigating inversion and improving worst-case latency.

References

- lab-2/LAB-2-REQUIREMENT.md
- QNX Momentics IDE 7.0 User's Guide/System-Architecture/Thread-Scheduling/Thread scheduling.md
- QNX Momentics IDE 7.0 User's Guide/System-Architecture/Thread-Scheduling/Scheduling policies.md
- QNX Momentics IDE 7.0 User's Guide/System-Architecture/IPC issues.md

- QNX Momentics IDE 7.0 User's Guide/System-Architecture/Synchronization-services/Synchronization services.md
- QNX Momentics IDE 7.0 User's Guide/System-Architecture/Synchronization-services/Semaphores.md